

## **EXPERIMENT 4: VACUUM CLEANER PROBLEM**

### **Aim:**

To develop a Python program that simulates the Vacuum Cleaner Problem in Artificial Intelligence by cleaning all dirty rooms using a simple reflex agent.

### **Objective:**

- To understand the concept of a Simple Reflex Agent in AI.
- To simulate the behaviour of a vacuum cleaner in a two-room environment.
- To clean all dirty rooms by sensing the room status.
- To stop the execution after both rooms become clean.

### **Algorithm:**

**Step 1:** Start.

**Step 2:** Initialize two rooms (A and B) as Dirty.

**Step 3:** Place the vacuum cleaner in Room A.

**Step 4:** Display the current location and room status.

**Step 5:** Check whether the current room is Dirty.

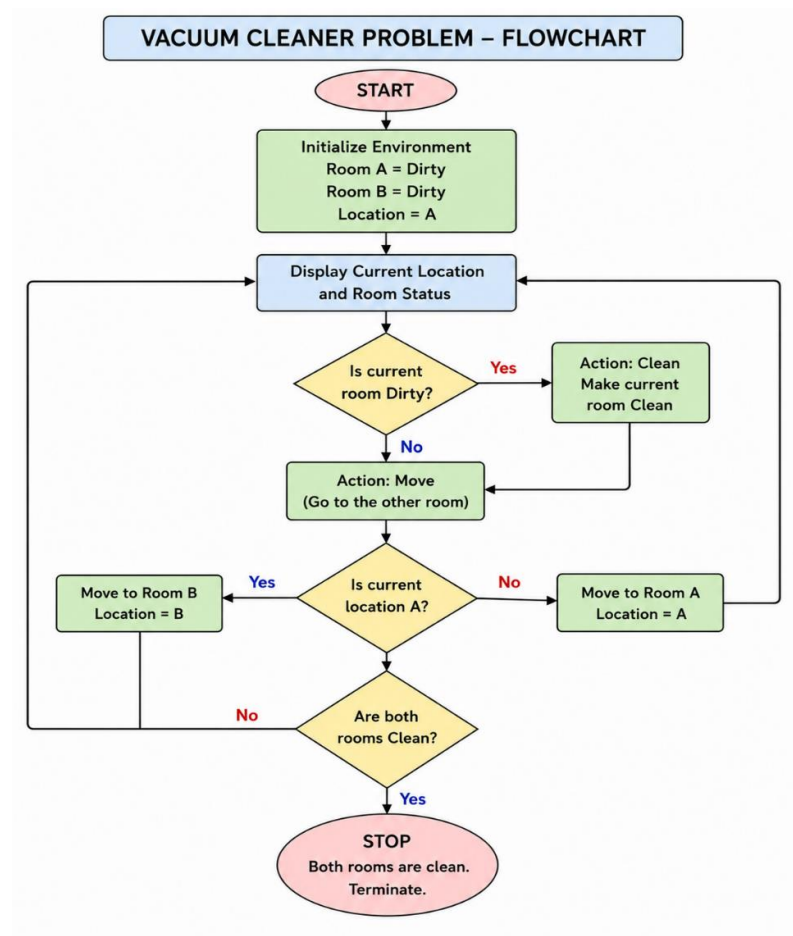
- If Yes, clean the room.
- If No, move to the other room.

**Step 6:** Check whether both rooms are Clean.

- If Yes, display "Both rooms are clean" and stop.
- If No, repeat from Step 4.

**Step 7:** Stop.

## Flow chart :



## Source code :

### Source code:

```
from collections import deque
```

```
goal = [[1, 2, 3],  
        [4, 5, 6],  
        [7, 8, 0]]
```

```
start = [[1, 2, 3],  
        [4, 0, 6],  
        [7, 5, 8]]
```

```
def board_to_tuple(board):  
    return tuple(tuple(row) for row in board)
```

```

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i, j

def get_neighbors(board):
    x, y = find_blank(board)
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
    neighbors = []

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:

            new_board = [row[:] for row in board]
            new_board[x][y], new_board[nx][ny] = new_board[nx][ny],
new_board[x][y]
            neighbors.append(new_board)

    return neighbors

def bfs(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        board, path = queue.popleft()

        if board == goal:
            return path + [board]

        state = board_to_tuple(board)
        if state in visited:
            continue

        visited.add(state)

```

```

        for neighbor in get_neighbors(board):
            queue.append((neighbor, path + [board]))

    return None

solution = bfs(start, goal)

if solution:
    print("Solution Found:\n")
    for step in solution:

        for row in step:
            print(row)
        print()
else:
    print("No Solution Found")

```

### Output :

```

Current Location: A
Room Status: {'A': 'Dirty', 'B': 'Dirty'}
Action: Clean

Current Location: A
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Move

Current Location: B
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Clean

Both rooms are clean.

Process finished with exit code 0
|

```

### Result:

The Python program successfully simulates the Vacuum Cleaner Problem. The vacuum cleaner cleans all dirty rooms and terminates after both rooms become clean.